Start Trek Reinforcement Learning Project Report

Martin Muller
Victor Zhou

May 2026

Contents

I Introduction **3**

I.1 Project Context **3**

I.2 Objectives **3**

II Machine Learning Theory **3**

II.1 Machine Learning **3**

II.2 Neural Networks **4**

II.3 Reinforcement Learning **4**

III Models Used **4**

III.1 What is a Model? **4**

III.2 DQN and Improvements **5**

III.3 Proximal Policy Optimization (PPO) **6**

IV Implementation **6**

IV.1 Code Implementation **6**

V Training **7**

V.1 Configuration **7**

V.2 Training Logs **7**

VI Results and Evaluation **9**

VI.1 Evaluation Metrics **9**

VI.2 Performance Comparison **9**

VII Conclusion **11**

VII.1 Conclusion **11**

References **11**

Introduction

I.1 Project Context

This report is part of the Start Trek Epitech project. The Start Trek is based on the Lunar Lander environment from OpenAI gymnasium.



I.2 Objectives

The objective of this project is to design an agent capable of solving the LunarLander task using reinforcement learning.

Beyond achieving a successful landing, we aim to answer a broader question: *can a learned model outperform human intuition and handcrafted strategies?*

To investigate this, we implemented two baselines:

- A **human-controlled version** of the environment
- A simple **heuristic agent** based on manually designed rules

These baselines provide a reference point against which learned models can be evaluated.

The report then explores different reinforcement learning methods and compares their performance to these baselines.

Machine Learning Theory

II.1 Machine Learning

Machine Learning is a field of artificial intelligence that focuses on designing algorithms capable of learning patterns from data without being explicitly programmed. Instead of relying on fixed

rules, machine learning models improve their performance by identifying relationships within data through a training process. There are three main categories of machine learning: supervised learning, unsupervised learning, and reinforcement learning. In supervised learning, the model learns from labeled data, while in unsupervised learning it identifies hidden structures in unlabeled data. Reinforcement learning, which is the focus of this project, differs significantly as it relies on interaction with an environment rather than static datasets.

II.2 Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks. These networks are composed of layers of interconnected neurons, where each neuron applies a linear transformation followed by a non-linear activation function. The power of deep learning comes from its ability to approximate complex functions through multiple layers. Each layer extracts higher-level features from the input data. In this project, neural networks are used to approximate value functions or policies, enabling the agent to make decisions based on the current state of the environment. Training a neural network consists of adjusting its parameters using optimization algorithms such as gradient descent, guided by a loss function that measures the difference between predictions and expected outcomes.

II.3 Reinforcement Learning

Reinforcement Learning (RL) is a learning paradigm where an agent interacts with an environment in order to maximize a cumulative reward. At each time step, the agent observes a state, selects an action, and receives a reward as feedback. The objective of the agent is to learn a policy, which is a function mapping states to actions, that maximizes the expected return over time. Unlike supervised learning, the agent does not receive explicit correct answers but must discover optimal behavior through trial and error. This interaction is typically modeled as a Markov Decision Process (MDP), defined by a set of states, actions, transition probabilities, and rewards.

A key challenge in reinforcement learning is balancing exploration and exploitation: the agent must explore new actions to discover better strategies while exploiting known actions that yield high rewards.

Models Used

III.1 What is a Model?

In this project, a **model** is a neural network that approximates a decision function. Instead of storing all state-action values explicitly, it learns to *generalize*: given a new state, it can estimate reasonable action values.

The input corresponds to the environment state (8 variables in LunarLander), and the output is a value for each possible action. The agent selects the action with the highest value.

Each model presented below improves upon the previous one by addressing a specific limitation. In contrast to learned policies, heuristic approaches rely on manually designed rules based on human intuition. While these methods can perform reasonably well, they are often limited in their ability to generalize and adapt to complex dynamics.

Reinforcement learning aims to overcome these limitations by learning optimal behavior directly from interaction with the environment.

III.2 DQN and Improvements

DQN

DQN approximates the action-value function $Q(s, a)$, defined by the Bellman equation:

$$Q^*(s, a) = E\left[r + \gamma \max_{a'} Q^*(s', a')\right] \tag{1}$$

Two mechanisms stabilize training: **experience replay**, which breaks temporal correlations, and a **target network**, which prevents moving targets during learning. Exploration is handled with an ε-greedy policy.

However, DQN suffers from **overestimation bias** due to the max operator and inefficient uniform sampling.

Double DQN

DDQN reduces overestimation by decoupling action selection and evaluation:

$$y = r + \gamma\, Q\Big(s',\, \arg\max_{a'} Q(s', a'; \theta);\ \theta^-\Big) \tag{2}$$

This simple modification significantly improves stability without changing the architecture.

Prioritized Experience Replay

PER improves sample efficiency by replaying important transitions more frequently. Each transition is sampled with probability:

$$P(i) = \frac{(|\delta_i| + \varepsilon)^\alpha}{\sum_k (|\delta_k| + \varepsilon)^\alpha} \tag{3}$$

where δ_i is the TD error. This accelerates learning but introduces additional hyperparameters.

Dueling Networks

The network is split into a value and an advantage stream:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')\right) \tag{4}$$

This allows the model to better represent states where actions have similar effects.

N-Step Returns

Instead of a 1-step update, rewards are accumulated over n steps:

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_{a'} Q(s_{t+n}, a') \tag{5}$$

This speeds up reward propagation but increases variance.

Noisy Networks

Exploration is learned by injecting noise into the network weights:

$$w = \mu + \sigma \cdot \varepsilon \tag{6}$$

This replaces ε-greedy with adaptive, state-dependent exploration.

Rainbow DQN

Rainbow combines all previous improvements and adds **distributional RL**, modeling the full return distribution:

$$Q(s,a) = \sum_{i=1}^{51} p_i(s,a)\, z_i \tag{7}$$

This richer representation leads to better performance but increases complexity.

III.3 Proximal Policy Optimization (PPO)

Unlike DQN-based methods, PPO directly learns a policy $\pi_\theta(a|s)$ instead of a value function. The objective is:

$$\nabla_\theta J(\theta) = E\left[\nabla_\theta \log \pi_\theta(a|s)\, A(s,a)\right] \tag{8}$$

To prevent unstable updates, PPO uses a clipped objective:

$$\mathcal{L}^{CLIP} = E\left[\min\left(r_t(\theta)A_t,\ clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right] \tag{9}$$

with

$$r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)} \tag{10}$$

PPO relies on an **actor-critic** architecture: the actor learns the policy, while the critic estimates $V(s)$ to compute the advantage.

This approach is more stable and naturally supports continuous actions, but it is less sample-efficient since it requires fresh data at each update.

Implementation

IV.1 Code Implementation

To see the code implementation, please check out the notebook here on Github: Jupyter Notebook The notebook detail how a neural network model is incorporated into the project architecture. In

the repository, you will also find how to launch a full training and a global evaluation.

Training

V.1 Configuration

For training, all models use the same configuration template:

```
LunarLander-x:
  fc1_nodes: [number of nodes]
  number_of_layer: [number of layer]
  algo:
    activation_function: [function availble in pytorch]
    optimizer: [function availble in pytorch]
  env:
    wind:
      enable_wind: false # default value
      wind_power: 0 # default value
      turbulence_power: 0 # default value
    gravity:
      gravity: -10.0 # default value
    fuel:
      enable_fuel: false
    continuous:
      enable_continuous: false # default value
  hypers:
    memory:
      replay_memory_size: [memory size]
      mini_batch_size: [batch size]
    epsilon:
      epsilon_init: 0.9
      epsilon_min: 0.01
      epsilon_decay: [epsilon reduction applied after each step]
    stop_on_mean_reward: [after acheiving this score to stop training]
    learning_rate_a: [learning rate]
```

The configuration file contains the gymnasium environment variables and the hyper-parameters for the model construction.

V.2 Training Logs

During training, we collected different data. We stored those as text files

```
05-05 11:01:03: Training starting...
0:00:17: Episode 100 | Mean(100): -159.66 | epsilon: 0.61
```

```
Termination stats at episode 100
```

```
=================================================
landed                    :     0 (   0.00%)
crash_high_speed          :    29 (  29.00%)
crash_bad_angle           :    36 (  36.00%)
out_of_bounds             :     6 (   6.00%)
time_limit                :     0 (   0.00%)
unknown                   :    29 (  29.00%)
=================================================
0:01:14: Episode 200 | Mean(100): -91.20 | epsilon: 0.37
=================================================
Termination stats at episode 200
=================================================
landed                    :     1 (   1.00%)
crash_high_speed          :    16 (  16.00%)
crash_bad_angle           :    18 (  18.00%)
out_of_bounds             :     4 (   4.00%)
time_limit                :     5 (   5.00%)
unknown                   :    56 (  56.00%)
=================================================
[...]
=================================================
0:31:42: Episode 1388 | Mean(100): 250.21 | epsilon: 0.01
=================================================
Termination stats at episode 1388
=================================================
landed                    :    60 (  68.18%)
crash_high_speed          :     0 (   0.00%)
crash_bad_angle           :     0 (   0.00%)
out_of_bounds             :     0 (   0.00%)
time_limit                :     0 (   0.00%)
unknown                   :    28 (  31.82%)
=================================================
Solved in 1388 episodes! With a score of 250.20720921802237
```

Here you can see an example of the log file for the DDQN Agent. We tracked

- The training time
- The mean reward on the last hundred episodes
- The epsilon
- The termination causes (landed, crash at high speed, crash with bad angle, went out of bounds of the simulation and time limit)

The training stops when a mean reward of 250 points is reached. All the logs are available in the Github repository:here

Here is a full training time lapse video : video

Results and Evaluation

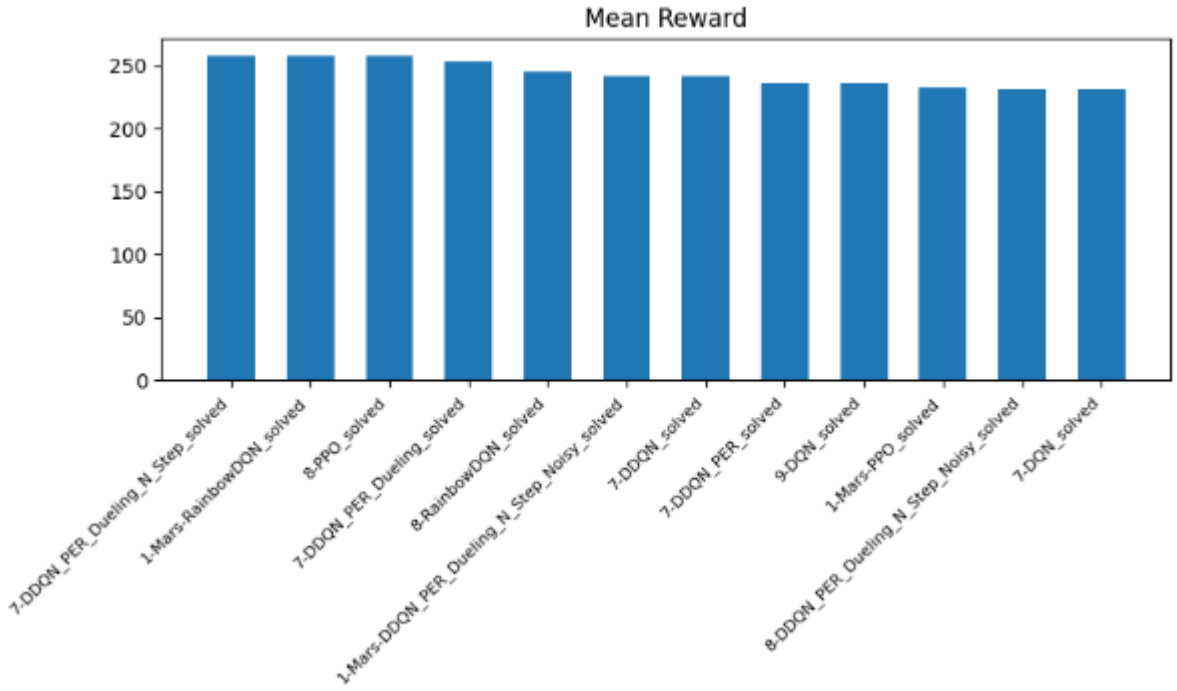
VI.1 Evaluation Metrics

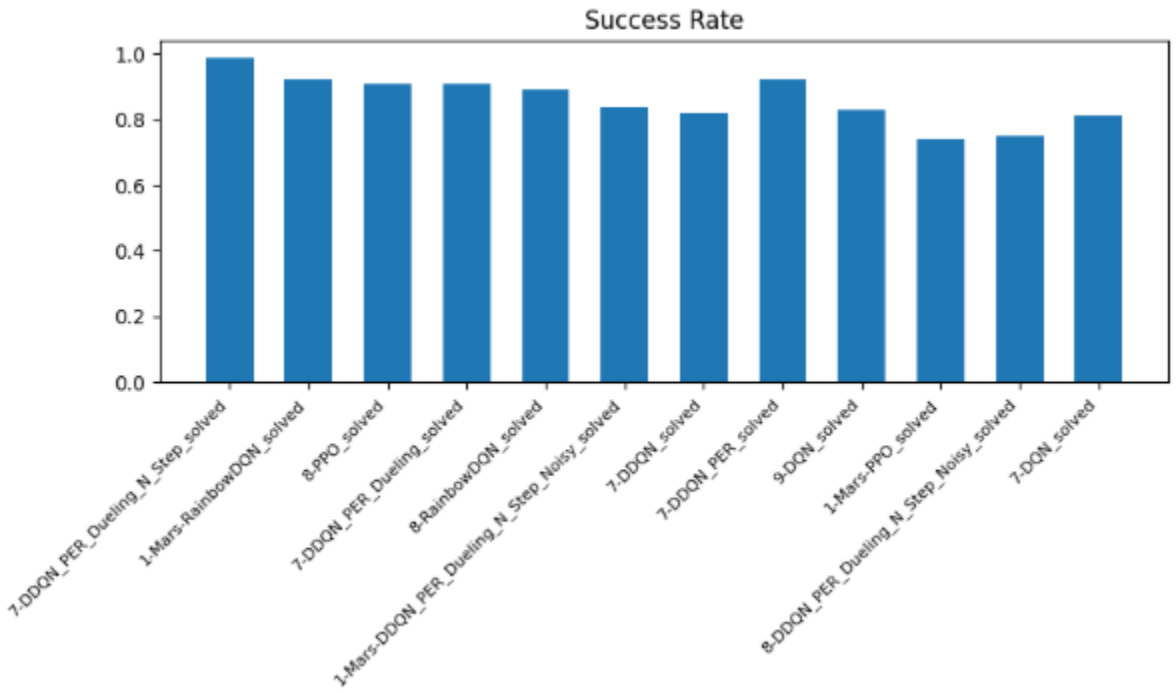
The objective is not only to have a working model. Since a simple DQN model can succeed with enough training, we needed to train and compare the results of different models with different configurations. The main metric to verify the quality of a model is the mean reward on several episodes. But with enough training even simple models can achieve it. So we also compare:

- The training time
- The number of episodes needed for training
- Minimum and Maximum reward on a single episode
- The success rate

VI.2 Performance Comparison

After all the models were trained, we wanted to compare and rank the best ones for this exercise. In order to do so, we created a simple python script that runs all the checkpoint files for 100 episodes on the same seed.





We extracted the mean reward and the success rate on the 100 episodes and simply make a ranking. Here is the top 3:

- 1 : DDQN PER Dueling N Step, 258.46 mean reward, 0.99 of success rate, 1 hour and 6 minutes of training
- 2 : PPO, 257.76 mean reward, 0.91 of success rate, 1 hour and 20 minutes of training
- 3 : DDQN PER Dueling, 253.88 mean reward, 0.91 of success rate, 1 hour and 29 minutes of training
- ...

In addition to comparing learning algorithms, we evaluated the performance of the human and heuristic baselines.

The heuristic agent was able to achieve stable but suboptimal landings, while our human performance varied significantly depending on reaction time and precision. Our testers were not able to make a successful landing.

```
0:01:11: Episode 7200 | Mean(100): -64.86 | epsilon: 0.00
```

```
Termination stats at episode 7200
```

landed	:	7	(7.00%)
crash_high_speed	:	40	(40.00%)
crash_bad_angle	:	2	(2.00%)
out_of_bounds	:	0	(0.00%)
time_limit	:	0	(0.00%)
unknown	:	51	(51.00%)

In contrast, trained reinforcement learning agents achieved:

- higher mean rewards,
- more consistent landings,
- and significantly higher success rates.

This demonstrates that learned policies can outperform both human intuition and handcrafted strategies in this environment.

After the test, the DDQN PER Dueling N Step obtained the best result with a mean superior to 250 and a almost perfect success rate of 0.99. It also has the fastest training time.

We wanted, for ethical concern, to calculate the electrical consumption of the best model. GPU consumption (kW) * Training time (h) = Energie (kWh)

For the DDQN PER Dueling N Step : 0.16 kW * 1,1 h = 0.17 (kWh) And since the 1 kWh produces 9 g of C02 in France : 0.17 kWh * 9 g = 1.53 g of CO2

Conclusion

VII.1 Conclusion

This project aimed not only to train reinforcement learning agents, but also to evaluate their performance against human intuition and heuristic approaches.
The results show that while heuristic and human-controlled strategies can achieve barely acceptable performance, they remain limited in consistency and optimality. In contrast, reinforcement learning agents—particularly advanced variants such as DDQN with PER and N-step returns—achieve higher rewards, near-perfect success rates, and more stable behavior.
This demonstrates the ability of learned models to surpass human-designed strategies in complex control tasks.
Beyond performance, this project highlights the importance of experimentation, evaluation protocols, and model comparison in reinforcement learning.
Finally, the adaptability of the trained agents to modified environments (e.g., Mars conditions) suggests strong generalization capabilities, reinforcing the relevance of these methods for real-world applications.

References

- Gymnasium Lunar Lander Documentation
- Reinforcement Learning
- PPO
- Rainbow DQN Statistic
- Rainbow DQN